

テクノロジー系試験範囲

1. 基礎理論

1-1. 基礎理論

離散数学/応用数学/情報に関する理論/通信に関する理論/継続・制御に関する理論

1-2. アルゴリズムとプログラミング

データ構造/アルゴリズム/プログラミング/プログラム言語/その他の言語

2. コンピュータシステム

2-1. コンピュータ構成要素

プロセッサ/メモリ/バス/入出力デバイス/入出力装置

2-2. システム構成要素

システムの構成/システムの評価指標

2-3. ソフトウェア

オペレーティングシステム/ミドルウェア/ファイルシステム/開発ツール/オープンソースソフトウェア

2-4. ハードウェア

ハードウェア

テクノロジー系試験範囲

3. 技術要素

3-1. ヒューマンインターフェース

ヒューマンインターフェース技術/インタフェース設計

3-2. マルチメディア

マルチメディア技術/マルチメディア応用

3-3. データベース

データベース方式/データベース設計/データ操作/トランザクション処理/データベース応用

3-4. ネットワーク

ネットワーク方式/データ通信と制御/通信プロトコル/ネットワーク管理/ネットワーク応用

3-5. セキュリティ

情報セキュリティ/情報セキュリティ管理/セキュリティ技術評価/情報セキュリティ対策/情報セキュリティ実装技術

4. 開発技術

4-1. システム開発技術

システム要件定義/システム方式設計/ソフトウェア要件定義/ソフトウェア方式設計・ソフトウェア詳細設計/ソフトウェア構築/ソフトウェア結合・ソフトウェア適格性確認テスト/ソフトウェア結合・ソフトウェア適格性確認テスト/システム結合・システム適格性確認テスト/導入/受入れ支援/保守・廃棄

4-2. ソフトウェア開発管理技術

開発プロセス・手法/知的財産権運用管理/開発環境管理/構成管理・変更管理

技術要素

データベース データベース方式、データベース設計

- 目標：**
- データベースの種類，特徴，データベースのモデル，3 層スキーマの考え方を修得する。
 - データベース管理システムの目的，機能を修得する。
 - データの分析の考え方を修得する。
 - データベースの設計の考え方，手順，手法を修得する。
 - データの正規化の目的，手順を修得する。
 - データベース作成の手順，評価方法を修得する。
 - オブジェクト指向データベースの考え方を修得する。

データベースとは

大量のデータを一定の規則に従って並べてデータを管理する

データベースの特徴

基本機能・・・データを挿入、削除、更新、検索を行う。

特性・・・原子性、一貫性、独立性、耐久性を持っていることが重要（ACID特性）

原子性	全ての処理が「すべて実行される」か「一つも実行されない」の状態。 例) 商品購入の場合、「商品を購入する」「在庫が減少する」という処理が同時に実行される
一貫性	処理前後でデータの整合性が保たれ、矛盾の無い状態が継続される性質。 例) 商品を複数個購入した場合に、商品の在庫数がマイナスにならないこと
独立性	実行中の処理が外部から隠蔽され、他の処理などに影響を与えない性質。 例) 商品購入の場合、商品の購入は実施しているが、在庫はまだ減っていないという状態が一時的に外部に見えないようにする（ 排他制御 ）
耐久性	処理が完了したら、結果は記録され、システム障害が生じても失われないこと。ログから復元出来ること

データベース管理システム(DBMS)

システムのデータベースを構築して、データを管理するためのシステム

関係（リレーショナル）データベース(RDMBS): 表形式でデータを管理するDBMS。Oracle Database, MySQL, PostgreSQL, SQLiteなどがあります。レコードは、対応するフィールドを指定して値を取り出したり、削除したり、更新したりできる。テーブル同士をフィールドで紐づけて結合できる。

テーブル

ユーザID	名前	性別	年齢	趣味
1	山田太郎	男	28	テニス
2	佐藤次郎	男	29 フィールド	ゴルフ
3	鈴木三郎	男	24	サッカー
4	田中花子	女	25	バレー

レコード

カラム

主キー: レコードを識別するためのカラム。重複があってははいけない。複数のカラムを組み合わせで主キーにすることもできる。

オブジェクト指向データベース・・・ 関係データモデルをオブジェクト指向データモデルに拡張し、操作の定義や型の継承関係の定義を可能にし、オブジェクトとしてデータを格納するデータベース

データベースの設計手順

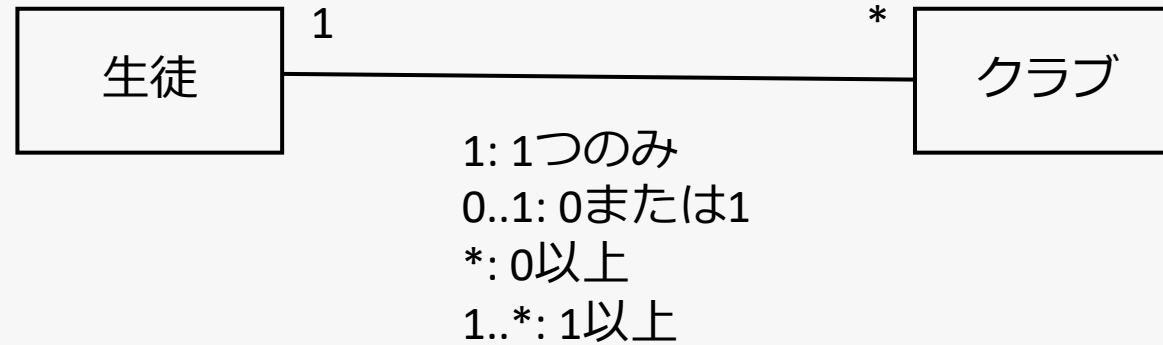
概念設計

論理設計

物理設計

概念設計・・・データベース化する対象となるデータを分析して洗い出す。
データは実体（エンティティ）とその関連（リレーションシップ）にまとめてE-R図で表します。

E-R図



↑の例だと生徒1に対して0以上のクラブが紐づいている

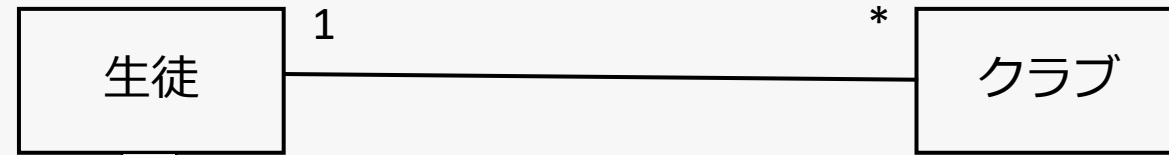
データベースの設計手順

概念設計

論理設計

物理設計

論理設計・・・概念設計の結果と実際の画面・帳票を用いて、データ構造を詳細化する。詳細化する際には**正規化**を行う(後述)。正規化をすることでデータの冗長性と不整合をなくす。



学籍番号	名前	年齢	性別
2020010001	吉田康二	15	男
2020010002	市川美子	16	女

:

物理設計・・・システム構成とデータベース製品を選定して、データベースの実装のための設計を行う。例えば、性能向上のためにインデックス構成を決めたり、データのレイアウト（重複を許すか、NULLを許すか等）、耐障害性、セキュリティを高めるための施策を考えます。

3層スキーマ

データベース設計時に意識する3つのデータの階層構造

概念スキーマ・・・データを論理的に構造化したもの。正規化されたスキーマ

外部スキーマ（サブスキーマ）・・・機密データを除いた外部に公開できるデータに限定したスキーマ

内部スキーマ・・・データファイル、インデックスなどをどのサーバのどこに配置するのかと言った。データベースを作成する際の物理的な構造

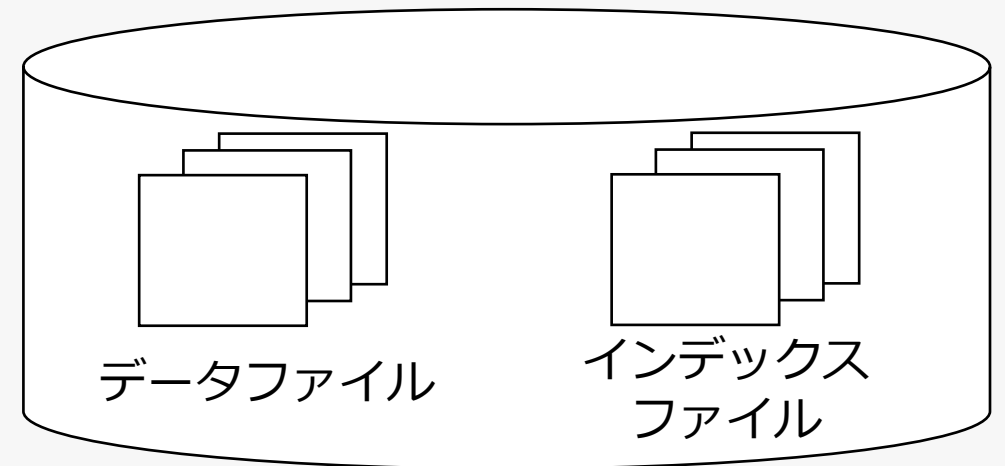
外部スキーマ

社員番号	名前	部署	年齢
------	----	----	----

概念スキーマ

社員番号	名前	部署	年齢	給料
------	----	----	----	----

内部スキーマ



応用情報技術者平成31年春期 午前問26

データベースを記録媒体にどのように格納するかを記述したものはどれか

ア. 概念スキーマ

→誤り。データの論理的な構造を表したものです

イ. 外部スキーマ

→誤り。機密データを除いた公開データのみにしたスキーマです

ウ. サブスキーマ

→誤り。サブスキーマは外部スキーマと同義です

エ. 内部スキーマ

→正しい。内部スキーマの説明です

基本情報技術者平成18年秋期 午前問41

外部設計工程の論理データ設計で行うべき作業項目はどれか。

ア. データ項目の洗い出しとデータ構造の決定

→正しい。論理(データ)設計でデータ構造を決定します。

イ. データファイル編成と媒体の決定

→誤り。物理設計です

ウ. データへのアクセス時間とデータファイルの容量の見積り

→誤り。物理設計です

エ. データレコードのレイアウト決定

→誤り。物理設計です

制約

候補キー: 行を一意に識別するために最小限必要な列の組み合わせ

主キー(PK): テーブル内でレコードを一意に識別することができるように指定するカラム。複数のカラムを指定することもできる。SQLで定義する際には、**PRIMARY KEY**と指定する。

外部キー(FK): 他のテーブルと紐づけるために用いるカラム。重複してもよく、複数のカラムを指定することもできる。SQLで定義する際には、**FOREIGN KEY, REFERENCES**で指定する。

インデックス: 索引のことでレコードを取得する際の高速化をしたい場合に利用される。

主キー制約

一意性制約	主キー（組み合わせも含む）は必ず、一意でなければならず、重複する行は存在しない
非ナル(NOT NULL) 制約	主キーの列は、空の値（NULL）をとってはならない。

学年	クラス	番号	名前	性別	年齢	...
1	A	12	菊池愛	女	14	...
2	B	13	中元鈴子	女	15	...

主キー ← 被ってはいけない、空の値を入れてはいけない

参照制約（外部キー制約）

外部キーの値が参照先に必ず存在すること。以下の2パターンでエラーとなる

- 参照されている行の削除
- 参照先のないものを参照している行の追加

CLUBSテーブル

ID	クラブ名	...
1	バスケット	...
2	テニス	...

越前良太に参照されているため、削除できない

STUDENTSテーブル

ID	名前	性別	クラブID	...
1	桜木花子	女	1	...
2	越前良太	男	2	...
3	上杉達志	男	3	...

外部キー

クラブIDが3のクラブは存在しないため、挿入できない

テーブル定義の際には、FOREIGN KEY, REFERENCESで指定する。FOREIGN_KEY (STUDENTS.クラブID) REFERENCES (CLUBS.ID)

その他の制約（チェック（検査）制約）：ある列が指定の条件を満たしているのかをチェックする制約（文字数が100文字以内等）。CHECK句を用いて宣言する

データの正規化

データを管理・整理しやすくするためのテーブル設計の作業。データの矛盾・重複がないようにする。第1正規形~第5正規形までであるが、基本情報・応用情報では第3正規形まで出題される

第1正規形: 繰返し属性（項目）を排除した形

同じ内容を繰り返している

学生番号	...	科目名	点数	...	科目名	点数
1001	...	英語	90	...	数学	80



学生番号	...	科目名	点数
1001	...	英語	90
1001	...	数学	80

第1正規形を満たしていない表を非正規形という

第2正規形: 候補キーの一部に従属する（部分関数従属）列が存在しないこと

候補キー

学生番号	科目コード	科目名	点数	...
1001	EN	英語	90	...
1001	MA	数学	80	...

科目名が科目コードに従属している



学生番号	科目コード	点数	...
1001	EN	90	...
1001	MA	80	...

テーブルを二つに分ける

科目コード	科目名	...
EN	英語	...
MA	数学	...

科目名をテーブルから削除して、新たに作成したテーブルで管理する

第3 正規形: 候補キー以外で、従属する（推移的関数従属）列が存在しないこと

候補キー

団体IDに従属している

資格コード	資格名	団体ID	団体名	...
FE	基本情報技術者試験	1234	IPA	...
AP	応用情報技術者試験	1234	IPA	...

テーブルを二つに分ける

資格コード	資格名	団体ID	...
FE	基本情報技術者試験	1234	...
AP	応用情報技術者試験	1234	...

団体ID	団体名
1234	IPA

団体名をテーブルから削除して、新たに作成したテーブルで管理する

基本情報技術者平成29年秋期 午前問27

SQL文においてFOREIGN KEYとREFERENCESを用いて指定する制約はどれか。

ア. キー制約

→誤り。キー制約は主キー制約と外部キー制約の総称で主キー制約の場合はPRIMARY KEYを用います

イ. 検査制約

→誤り。CHECK句を用います

ウ. 参照制約

→正しい。参照制約はFOREIGN KEYとREFERENCESです

エ. 表明

→誤り。複数のテーブルに対して制約をかけます。CREATE ASSERTIONを用います。

応用情報技術者平成29年春期 午前問27

"受注明細"表は、どのレベルまでの正規形の条件を満足しているか。ここで、実線の下線は主キーを表す。

受注明細

<u>受注番号</u>	<u>明細番号</u>	商品コード	商品名	数量
015867	1	TV20006	20 型テレビ	20
015867	2	TV24005	24 型テレビ	10
015867	3	TV28007	28 型テレビ	5
015868	1	TV24005	24 型テレビ	8

ア. 第1正規形

イ. 第2正規形

ウ. 第3正規形

エ. 第4正規形

→繰り返し列はないため、第1正規形は満足しています。

主キーに従属する（部分従属）する列もないため第2正規形も満足しています。

商品名は、商品コードに従属しています（推移的関数従属）。そのため、第3正規形は満たしていません。**よってイ. 第2正規形**まで満足しています

まとめ

データベースの特徴・・・データを挿入、削除、更新、検索ができること

ACID特性・・・原子性、一貫性、独立性、耐久性

関係データベース・・・テーブル、レコード、カラム、フィールド

オブジェクト指向データベース

設計手順・・・概念設計、論理設計、物理設計

3層スキーマ・・・概念スキーマ、外部スキーマ、内部スキーマ

制約・・・候補キー、主キー、外部キー、インデックス、一意性制約、非ナル制約、参照制約（外部キー制約）、チェック制約

データの正規化・・・第1正規形（繰返し属性）、第2正規形（部分関数従属）、第3正規形（推移的関数従属）

過去問（データベース方式、データベース
設計）

「出典：応用情報技術者令和4年秋期 午前問26」

データ項目の命名規約を設ける場合、次の命名規約だけでは回避できない事象はどれか。

〔命名規約〕

- (1): データ項目名の末尾には必ず"名", "コード", "数", "金額", "年月日"などの区分語を付与し, 区分語ごとに定めたデータ型にする。
- (2): データ項目名と意味を登録した辞書を作成し, 異音同義語や同音異義語が発生しないようにする。

ア. データ項目"受信年月日"のデータ型として, 日付型と文字列型が混在する。

イ. データ項目"受注金額"の取り得る値の範囲がテーブルによって異なる。

ウ. データ項目"賞与金額"と同じ意味で"ボーナス金額"というデータ項目がある。

エ. データ項目"取引先"が, "取引先コード"か"取引先名"か, 判別できない。

「出典：応用情報技術者令和4年秋期 午前問26」

データ項目の命名規約を設ける場合、次の命名規約だけでは回避できない事象はどれか。

〔命名規約〕

- (1): データ項目名の末尾には必ず"名", "コード", "数", "金額", "年月日"などの区分語を付与し、区分語ごとに定めたデータ型にする。
- (2): データ項目名と意味を登録した辞書を作成し、異音同義語や同音異義語が発生しないようにする。

ア. データ項目"受信年月日"のデータ型として、日付型と文字列型が混在する。

→誤り。命名規約(1)により、「"年月日"」という区分語が末尾に付いている場合、特定のデータ型にすることが定められています。これにより、データ型が混在することは回避されます。

イ. データ項目"受注金額"の取り得る値の範囲がテーブルによって異なる。

→正しい。標準化は規定されていますが、データ項目の値の範囲に関する規定は含まれていません。したがって、この問題は命名規約だけでは回避できない事象です。

ウ. データ項目"賞与金額"と同じ意味で"ボーナス金額"というデータ項目がある。

→誤り。命名規約(2)により、異音同義語や同音異義語が発生しないようにデータ項目名と意味を登録した辞書を作成することが定められています。

エ. データ項目"取引先"が、"取引先コード"か"取引先名"か、判別できない。

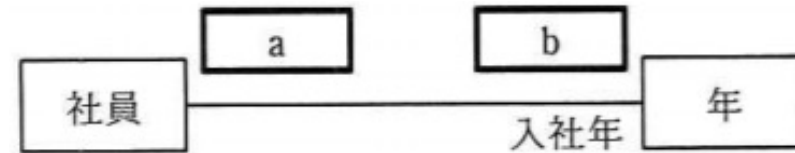
→誤り。命名規約(1)により、データ項目名の末尾には区分語を付与することが定められています。

「出典：応用情報技術者平成30年秋期 午前問26」

社員と年の対応関係をUMLのクラス図で記述する。二つのクラス間の関連が次の条件を満たす場合、a, bに入れる多重度の適切な組合せはどれか。ここで、"年"クラスのインスタンスは毎年存在する。

〔条件〕

- (1): 全ての社員は入社年を特定できる。
- (2): 年によっては社員が入社しないこともある。



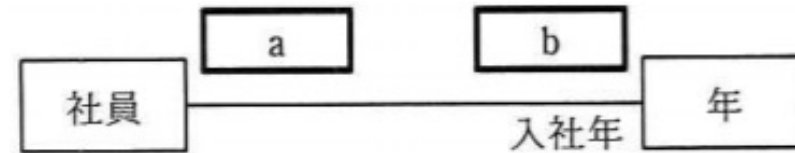
	a	b
ア	0..*	0..1
イ	0..*	1
ウ	1..*	0..1
エ	1..*	1

「出典：応用情報技術者平成30年秋期 午前問26」

社員と年の対応関係をUMLのクラス図で記述する。二つのクラス間の関連が次の条件を満たす場合、a, bに入れる多重度の適切な組合せはどれか。ここで、"年"クラスのインスタンスは毎年存在する。

〔条件〕

- (1): 全ての社員は入社年を特定できる。
- (2): 年によっては社員が入社しないこともある。



	a	b
ア	0..*	0..1
イ	0..*	1
ウ	1..*	0..1
エ	1..*	1

→

a: (2)より年によっては社員が入社しない場合もあるので、0以上の社員が各、年に対して紐づいている。そのため、0..*であるア、イが正解

b: (1)よりすべての社員は入社年を特定できるため、各社員には年が1つ紐づく。そのため、bは1であり、

イが正解

「出典：応用情報技術者平成26年春期 午前問27」

次の表はどこまで正規化されたものか。

従業員番号	氏名	入社年	職位	職位手当
12345	情報 太郎	1981	部長	90,000
12346	処理 次郎	1995	課長	50,000
12347	技術 三郎	1997	課長	50,000

ア. 第2正規形

イ. 第3正規形

ウ. 第4正規形

エ. 非正規形

「出典：応用情報技術者平成26年春期 午前問27」

次の表はどこまで正規化されたものか。

従業員番号	氏名	入社年	職位	職位手当
12345	情報 太郎	1981	部長	90,000
12346	処理 次郎	1995	課長	50,000
12347	技術 三郎	1997	課長	50,000

ア. 第2正規形

イ. 第3正規形

ウ. 第4正規形

エ. 非正規形

→繰り返し列はないため、第1正規化はされています。
候補キーは従業員番号ですべて一意に決定でき、また、他の列は従業員番号に従属していません。そのため、第2正規化はされています。
職位手当は、職位に紐づいているため、第3正規化はされていません。
よって正解は**ア. 第2正規形**です

「出典：基本情報技術者平成28年春期 午前問29」

関係データベースにおいて、外部キーを定義する目的として、適切なものはどれか。

ア.関係する相互のテーブルにおいて、レコード間の参照一貫性が維持される制約をもたせる。

イ.関係する相互のテーブルの格納場所を近くに配置することによって、検索、更新を高速に行う。

ウ.障害によって破壊されたレコードを、テーブル間の相互の関係から可能な限り復旧させる。

エ.レコードの削除、追加の繰返しによる、レコード格納エリアのフラグメンテーションを防止する。

「出典：基本情報技術者平成28年春期 午前問29」

関係データベースにおいて、外部キーを定義する目的として、適切なものはどれか。

ア.関係する相互のテーブルにおいて、レコード間の参照一貫性が維持される制約をもたせる。

→正しい。外部キーを定義することで、参照一貫性（外部キーの値が参照先に必ず存在すること）を維持できます。

イ.関係する相互のテーブルの格納場所を近くに配置することによって、検索、更新を高速に行う。

→誤り。外部キーと格納場所とは無関係です

ウ.障害によって破壊されたレコードを、テーブル間の相互の関係から可能な限り復旧させる。

→誤り。外部キーの定義とは関係がありません。

エ.レコードの削除、追加の繰返しによる、レコード格納エリアのフラグメンテーションを防止する。

→外部キーがあっても、レコード削除時にはフラグメンテーションは防ぐことはできません

「出典：基本情報技術者平成30年秋期 午前問27」

データ項目の命名規約を設ける場合、次の命名規約だけでは回避できない事象はどれか。

〔命名規約〕

- (1): データ項目名の末尾には必ず"名", "コード", "数", "金額", "年月日"などの区分語を付与し, 区分語ごとに定めたデータ型にする。
- (2): データ項目名と意味を登録した辞書を作成し, 異音同義語や同音異義語が発生しないようにする。

ア.データ項目"受信年月日"のデータ型として, 日付型と文字列型が混在する。

イ.データ項目"受注金額"の取り得る値の範囲がテーブルによって異なる。

ウ.データ項目"賞与金額"と同じ意味で"ボーナス金額"というデータ項目がある。

エ.データ項目"取引先"が, "取引先コード"か"取引先名"か, 判別できない。

「出典：基本情報技術者平成30年秋期 午前問27」

データ項目の命名規約を設ける場合、次の命名規約だけでは回避できない事象はどれか。

〔命名規約〕

- (1): データ項目名の末尾には必ず"名", "コード", "数", "金額", "年月日"などの区分語を付与し、区分語ごとに定めたデータ型にする。
- (2): データ項目名と意味を登録した辞書を作成し、異音同義語や同音異義語が発生しないようにする。

ア.データ項目"受信年月日"のデータ型として、日付型と文字列型が混在する。

→区分語ごとにデータ型を定めるため、日付型と文字列型が混在することはないです

イ.データ項目"受注金額"の取り得る値の範囲がテーブルによって異なる。

→正しい。データ型については定めていますが、データのとりうる範囲は定めていません

ウ.データ項目"賞与金額"と同じ意味で"ボーナス金額"というデータ項目がある。

→異音同義語や同音異義語が発生しないようにするため、この事象は回避できます

エ.データ項目"取引先"が、"取引先コード"か"取引先名"か、判別できない。

→データ項目名の末尾には必ず"名", "コード", "数", "金額", "年月日"をつけるため、"取引先コード"か"取引先名"は区別できます。

テクノロジー系試験範囲

1. 基礎理論

1-1. 基礎理論

離散数学/応用数学/情報に関する理論/通信に関する理論/継続・制御に関する理論

1-2. アルゴリズムとプログラミング

データ構造/アルゴリズム/プログラミング/プログラム言語/その他の言語

2. コンピュータシステム

2-1. コンピュータ構成要素

プロセッサ/メモリ/バス/入出力デバイス/入出力装置

2-2. システム構成要素

システムの構成/システムの評価指標

2-3. ソフトウェア

オペレーティングシステム/ミドルウェア/ファイルシステム/開発ツール/オープンソースソフトウェア

2-4. ハードウェア

ハードウェア

テクノロジー系試験範囲

3. 技術要素

3-1. ヒューマンインターフェース

ヒューマンインターフェース技術/インタフェース設計

3-2. マルチメディア

マルチメディア技術/マルチメディア応用

3-3. データベース

データベース方式/データベース設計/**データ操作/トランザクション処理/データベース応用**

3-4. ネットワーク

ネットワーク方式/データ通信と制御/通信プロトコル/ネットワーク管理/ネットワーク応用

3-5. セキュリティ

情報セキュリティ/情報セキュリティ管理/セキュリティ技術評価/情報セキュリティ対策/情報セキュリティ実装技術

4. 開発技術

4-1. システム開発技術

システム要件定義/システム方式設計/ソフトウェア要件定義/ソフトウェア方式設計・ソフトウェア詳細設計/ソフトウェア構築/ソフトウェア結合・ソフトウェア適格性確認テスト/ソフトウェア結合・ソフトウェア適格性確認テスト/システム結合・システム適格性確認テスト/導入/受入れ支援/保守・廃棄

4-2. ソフトウェア開発管理技術

開発プロセス・手法/知的財産権運用管理/開発環境管理/構成管理・変更管理

技術要素

データベース

データ操作、トランザクション処理、データベース応用

- 目標：**
- 関係データベースのデータの操作を修得する。
 - データベース言語の種類，SQL 文を修得する。
 - データベースの同時実行制御（排他制御），障害回復の考え方，仕組みを修得する。
 - トランザクション管理，アクセス効率向上のための考え方を修得する。
 - データに対するアクセス制御の必要性，代表的なアクセス権限を修得する。
 - データベースの応用対象，応用方法を修得する。
 - 分散データベース及び NoSQL の特徴，機能を修得する。
 - データ資源管理の仕組みとして，リポジトリ，データディクショナリを修得する。

データ操作

DBMSのデータ操作の代表的なデータ操作を以下に上げる

選択	テーブルから必要なレコードだけ取り出す
挿入	データを挿入する
削除	データを削除する
射影	データの一部のカラムだけを抽出する
結合操作	複数のテーブルを結合して抽出する

SELECT文

左の表から、生徒全体を取り出す

SELECT SELECTでデータを取り出す

* ←*: すべてのデータ

FROM students;

FROM ○○でデータを取り出す対象
のテーブルを指定する

studentsテーブル

student_no	name	age	sex
1	山田太郎	15	男
2	佐藤次郎	16	男

データ挿入(INSERT)、更新(UPDATE)、削除(DELETE)

INSERT文を用いるとデータを挿入する

INSERT INTO students VALUES(7, '鈴木五郎', 19, 3); # studentsテーブルに、student_no:7, name:鈴木五郎, age:19, club_no:3の行を追加

INSERT INTO students(student_no, name, club_no) VALUES(7, '鈴木五郎', 3); # studentsテーブルに、student_no:7, name:鈴木五郎, club_no:3の行を追加。ageは値を指定していないため、デフォルト値がNULL

UPDATE文でデータを更新

UPDATE students SET name='石井花子' WHERE student_no = 6; # studentsテーブルからstudent_noが6の行のnameを石井花子にする

UPDATE students set name = '石井太郎'; # studentsテーブルのnameを全て石井太郎にする

DELETE文でデータを削除

DELETE FROM students where student_no=7;
student_noが7の行を削除

DELETE FROM students;
studentsテーブルの行を全削除

studentsテーブル

student_no	name	age	club_no
1	山田太郎	15	1
2	佐藤次郎	16	2

べき等

同じ操作を複数回実行しても、結果が変わらない性質を指す。UPDATE 文で主キーを指定して更新する場合、同じ UPDATE 文を複数回実行しても、結果は変わらない（べき等）。同じ INSERT 文を複数回実行すると、重複したデータが挿入される（べき等でない）

WHERE句: 行を絞り込む

右の表から、student_noが1の生徒を取り出す

SELECT

*

FROM students

WHERE

student_no = 1

studentsテーブル

student_no	name	age	sex
1	山田太郎	15	男
2	佐藤次郎	16	男

WHERE句でstudent_noが1のものだけ取り出す
ANDとORでかつ、またはの複数条件の指定ができる

列名 = ○, 列名 <> ○, 列名 > ○, 列名 < ○	○と等しい。○以外。○より大きい。○より小さい
列名 BETWEEN ○○ AND ××	値○○以上××以下。*) student_no BETWEEN 1 AND 100
列名 NOT BETWEEN ○○ AND ××	値○○未満、××より大きい。*) student_no NOT BETWEEN 1 AND 100
列名 IN (x, y, z,...)	値がリストのいずれか。*) student_no IN(1, 2, 3)
列名 NOT IN(x, y, z,...)	値がリスト以外。*) student_no NOT IN(1,2,3)
列名 IS NULL	値がNULLのもの。*) name IS NULL
列名 IS NOT NULL	値がNULL以外のもの。*) name IS NOT NULL
列名 LIKE パターン	*) name LIKE '山田%' nameが山田で始まる。name LIKE '%太郎' nameが太郎で終わる。 name LIKE '%田%' nameに田があるもの
列名 NOT LIKE パターン	*) name NOT LIKE '山田%' nameが山田で始まらない。name NOT LIKE '%太郎' nameが太郎で終わらない。name NOT LIKE '%田%' nameに田がないもの

DISTINCT句: 重複行を1つにする

重複しているものをまとめる

SELECT

DISTINCT name

nameの重複
を削除

FROM students;

山田太郎
佐藤次郎
田中三郎
後藤四郎

SELECT

DISTINCT name, age

name,ageの重複を削除

FROM students;

studentsテーブル

student_no	name	age	sex
1	山田太郎	15	男
2	佐藤次郎	16	男
3	田中三郎	16	男
4	後藤四郎	14	男
6	山田太郎	15	男
7	佐藤次郎	18	男

山田太郎	15
佐藤次郎	16
田中三郎	16
後藤四郎	14
佐藤次郎	18

SELECT

name AS student_name # student_nameという列名をつけて取り出す

FROM students;

ORDER BY句: 行を並び替える

SELECT

*

FROM students

ORDER BY age (ASC);

ageで昇順に
並び替える



4	後藤四郎	14	男
1	山田太郎	15	男
5	山田太郎	15	男
2	佐藤次郎	16	男
3	田中三郎	16	男
6	佐藤次郎	18	男

studentsテーブル

student_no	name	age	sex
1	山田太郎	15	男
2	佐藤次郎	16	男
3	田中三郎	16	男
4	後藤四郎	14	男
5	山田太郎	15	男
6	佐藤次郎	18	男

SELECT

*

FROM students

ORDER BY age DESC;

ageで降順に並び替える

6	佐藤次郎	18	男
3	田中三郎	16	男
2	佐藤次郎	16	男
5	山田太郎	15	男
1	山田太郎	15	男
4	後藤四郎	14	男

ORDER BY age, nameとするとageとnameの複数列で昇順にする

GROUP BY句: 一部の列を用いて表をまとめる

```
SELECT
name, SUM(salary), MIN(salary), AVG(salary)
FROM students
```

GROUP BY name; nameでまとめる



山田太郎	240	100	120
佐藤次郎	280	120	140
田中三郎	150	150	150
後藤四郎	110	110	110

```
SELECT
name, SUM(salary), MIN(salary), AVG(salary)
FROM students
GROUP BY name
```

HAVING AVG(salary) >= 140;

AVG(salary)が140以上のものだけ取り出す

佐藤次郎	280	120	140
田中三郎	150	150	150

studentsテーブル

student_no	name	age	sex	salary
1	山田太郎	15	男	100
2	佐藤次郎	16	男	120
3	田中三郎	16	男	150
4	後藤四郎	14	男	110
5	山田太郎	15	男	140
6	佐藤次郎	18	男	160

集合関数

COUNT()	行数
SUM()	値の合計値
AVG()	値の平均値
MAX()	値の最大値
MIN()	値の最小値

副問い合わせ:SELECT文の中にSELECTを記載して絞込みに利用する

SELECT

*

FROM students

nameが東高校のschool_noの
studentsだけ抽出

WHERE school_no IN (
SELECT school_no FROM schools where name = '東高校')



1	山田太郎	15	1
2	佐藤次郎	16	1
6	佐藤次郎	18	1

SELECT

*

FROM students

nameが東高校以外のschool_noの
studentsだけ抽出

WHERE school_no NOT IN (
SELECT school_no FROM schools where name = '東
高校')



3	田中三郎	16	2
4	後藤四郎	15	2
5	山田太郎	16	3

studentsテーブル

student_no	name	age	school_no
1	山田太郎	15	1
2	佐藤次郎	16	1
3	田中三郎	16	2
4	後藤四郎	14	2
5	山田太郎	15	3
6	佐藤次郎	18	1

schoolsテーブル

school_no	name
1	東高校
2	西高校
3	北高校

副問い合わせ:existsを利用して存在するものだけ取り出す

SELECT

*

FROM students

WHERE EXISTS (
SELECT * FROM schools where name = '東高校' AND
school_no = students.school_no);

存在するもの
(student_no:1,2,6)だけ取
り出す

列名は何でも良い

studentsのschool_noが1のものだけ

取り出される



1	山田太郎	15	1
2	佐藤次郎	16	1
6	佐藤次郎	18	1

SELECT

*

FROM students

WHERE NOT EXISTS (
SELECT * FROM schools where name = '東高校'
AND school_no = students.school_no);

存在しないものだけ取り出す



3	田中三郎	16	2
4	後藤四郎	15	2
5	山田太郎	16	3

studentsテーブル

student_no	name	age	school_no
1	山田太郎	15	1
2	佐藤次郎	16	1
3	田中三郎	16	2
4	後藤四郎	14	2
5	山田太郎	15	3
6	佐藤次郎	18	1

schoolsテーブル

school_no	name
1	東高校
2	西高校
3	北高校

Viewの作成: テーブルからの問い合わせを仮想的に格納する(外部スキーマ)

```
CREATE VIEW v_students_over_15 AS  
SELECT  
    student_no, name  
FROM students WHERE age > 15;
```

studentsテーブルからageが15より大きいものを
取り出した表(v_students_over_15を作成)



更新可能なビュー: ビューから元のテーブルを更新できるビュー以下の条件がある

1. FROM句の表は1つしかないこと
2. WHERE句に副問い合わせがないこと
3. GROUP BYがないこと
4. DISTINCTや算術演算子(MAX等)を利用していないこと

テーブル定義:CREATE文を用いてテーブルを作成します

CREATE TABLE students(
student_no INTEGER PRIMARY KEY
name VARCHAR(256) NOT NULL,
age INTEGER CHECK(age >= 0) DEFAULT 0,
school_no INTEGER,
FOREIGN KEY (school_no)
REFERENCES schools (school_no)
);

テーブル名

データ型が数値(INTEGER)で主キー(PRIMARY KEY)
データ型がマックス255の文字列(VARCHAR)で非ナ
ル(NOT NULL)

列名

age>=0のチェックでデフォルト値が0

school_no列は、schoolsテーブルのschool_noへの外
部キー

制約一覧

PRIMARY KEY	主キー制約
NOT NULL	非ナル(NOT NULL)制約。
UNIQUE	一意性制約。同じ値は禁止する
CHECK	チェック(検査)制約
FOREIGN KEY REFERENCES	参照制約。参照制約のあるデータに紐づくデータも含め強制的に削除するには ON DELETE CASCADE をDELETE文と一緒に記載します

GRANT文,REVOKE文:テーブルへのアクセス権の付与・はく奪

GRANT **ALL PRIVILEGES** ON students TO USER1 # **USER1**にstudentsテーブルへの全アクセス権を与える

GRANT **SELECT, UPDATE** ON students TO USER1 # **USER1**にstudentsテーブルへのSELECT, UPDATE権を与える

REVOKE ALL PRIVILEGES ON students to USER2 # **USER2**からstudentsテーブルへの全アクセス権をはく奪する

アクセス権には、INSERT(挿入), UPDATE(更新), SELECT(問い合わせ), DELETE(削除)がある

SQLの関数

CONCAT	文字列を連結する。 例: CONCAT('Hello', ' ', 'World') の結果は 'Hello World'
SUBSTRING	文字列の一部を抽出する。 例: SUBSTRING('Hello World', 1, 5) の結果は 'Hello'。ここでは、1番目の文字から5文字分を抽出している。
LENGTH	LENGTH関数は文字列の長さを返す。 例: LENGTH('Hello World') の結果は 11。
TRIM	TRIM関数は文字列の先頭と末尾にある空白文字を削除する。 例: TRIM(' Hello World ') の結果は 'Hello World'
COALESCE	COALESCE関数は、NULL以外の最初の値を返す。 例: COALESCE(NULL, NULL, 'Hello', 'World') の結果は 'Hello'
ROUND	ROUND関数は数値を指定した桁数で丸める。 例: ROUND(123.456, 2) の結果は 123.46。小数点以下2桁で丸めている。
FLOOR	FLOOR関数は数値を切り捨てる。 例: FLOOR(123.456) の結果は 123
CEIL	CEIL関数は数値を切り上げる。 例: CEIL(123.456) の結果は 124

応用情報技術者平成31年春期 午前問27

RDBMSにおいて、特定の利用者だけに表を更新する権限を与える方法として、適切なものはどれか。

ア. CONNECT文で接続を許可する。

→CONNECTはOracleなどでDBに接続する際に用います

イ. CREATE ASSERTION文で表明して制限する。

→CREATE ASSERTIONは複数の表に対する制約です

ウ. CREATE TABLE文の参照制約で制限する。

→参照制約はテーブル間の制約であり、特定の利用者への制約にはなりません。

エ. GRANT文で許可する。

→正しい。GRANT文で特定の利用者に更新権限を与えます

基本情報技術者平成31年春期 午前問27

"中間テスト"表からクラスごと、教科ごとの平均点を求め、クラス名、教科名の昇順に表示するSQL文中のaに入れる字句はどれか。

中間テスト(クラス名,教科名,学生番号,名前,点数)

〔SQL文〕

```
SELECT クラス名, 教科名, AVG(点数) AS 平均点  
FROM 中間テスト a
```

ア. GROUP BY クラス名, 教科名 ORDER BY クラス名, AVG(点数)

イ. GROUP BY クラス名, 教科名 ORDER BY クラス名, 教科名

ウ. GROUP BY クラス名, 教科名, 学生番号 ORDER BY クラス名, 教科名, 平均点

エ. GROUP BY クラス名, 平均点 ORDER BY クラス名, 教科名

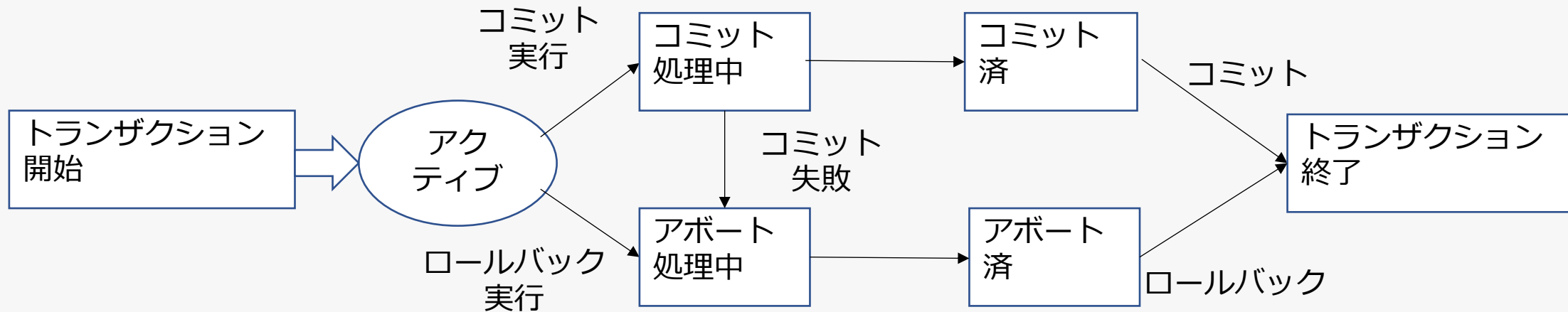
→正解はイ

GROUP BY クラス名, 教科名でクラス名、教科名ごとにまとめて平均点を算出します。

ORDER BY クラス名, 教科名でクラス名, 教科名を昇順に並べます(DESCを付けると降順)

トランザクション処理

一連の処理が全て成功したら処理結果を確定（**コミット**）し、途中で失敗したら処理前の状態に戻す（**ロールバック**）処理。トランザクションの結果を破棄してロールバックすることをアボートとも言います。



- ・ トランザクション処理が完了したら、コミットかロールバックを実行します
- ・ コミットが成功したらコミット済に移動して、失敗したらアボート処理される
- ・ アボート処理が成功したら、アボート済に移動する

障害回復(トランザクションの回復)

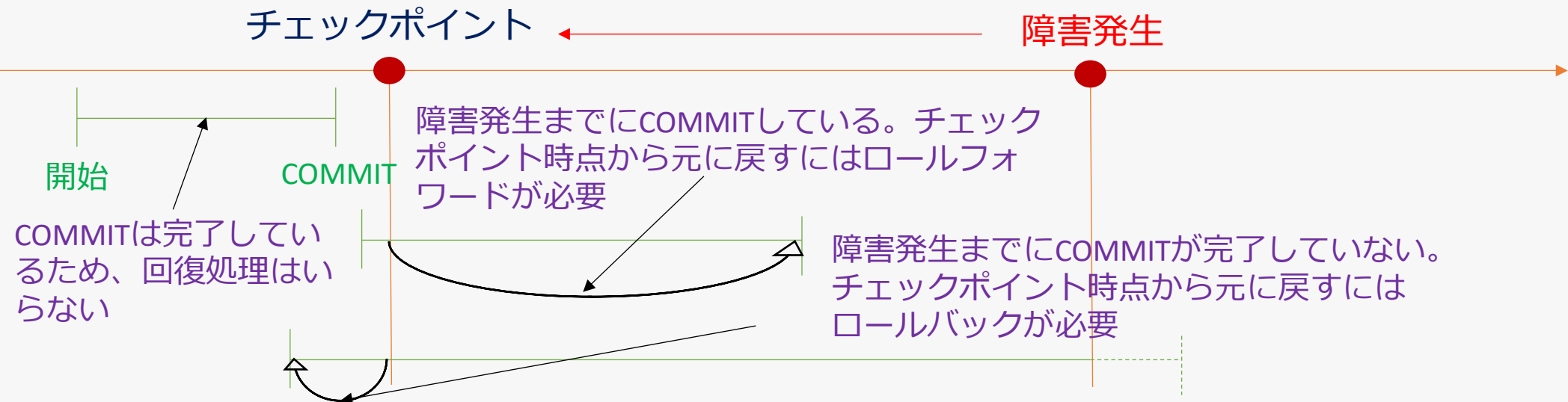
データベースのDBMS障害発生時にデータの矛盾がないよう障害直前の状態に戻ることが求められる。

ログ・・・DBに対して更新を行った記録を保管する。実行した更新を取り消すこと(ロールバック)ができるように実行前の状態を保持するログ (**UNDOログ**)。更新の途中で障害が発生した場合に、その時点までの状態に戻すこと(ロールフォワード)ができるように更新内容を記録するログ (**REDOログ**)。UNDOログ、REDOログはチェックポイント、COMMIT時に出力される。

チェックポイント・・・COMMITをデータベース(ハードディスク)に書き込む。COMMITの時点ではハードディスクに書き込まれておらず、COMMITをいくらかメモリ上に溜まったのちにハードディスクに書き込まれる

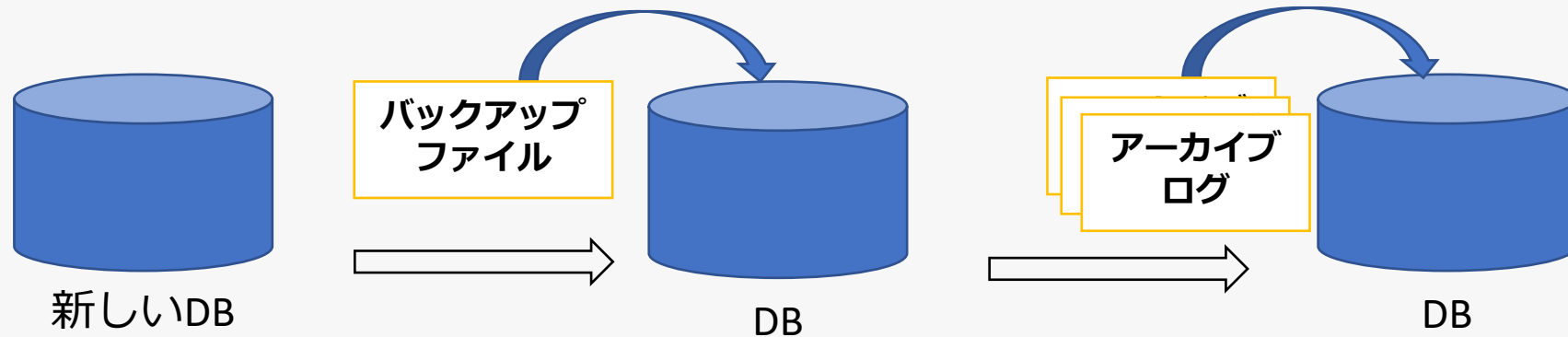
ロールバック・・・障害発生時に、UNDOログの内容を用いて、更新処理実行前の状態に戻す処理

ロールフォワード・・・障害発生時に、REDOログの内容を用いて、更新後の状態に戻す処理



障害回復(媒体障害からの回復)

データベースの本体（ストレージ）が壊れた場合、以下の手順で回復する



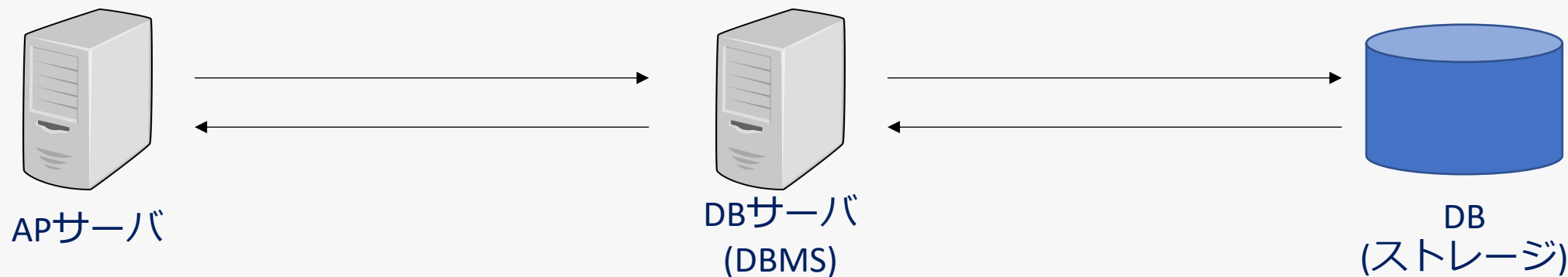
1. 新しいデータベースと交換する
2. **バックアップファイル**を用いて、バックアップ取得時点まで戻す
3. **アーカイブログ**を用いて、バックアップ取得時点から障害発生時にまで戻す（**ロールフォワード**）

バックアップファイル・・・取得した時点のデータベースの状態を記録するファイル（フルバックアップ）

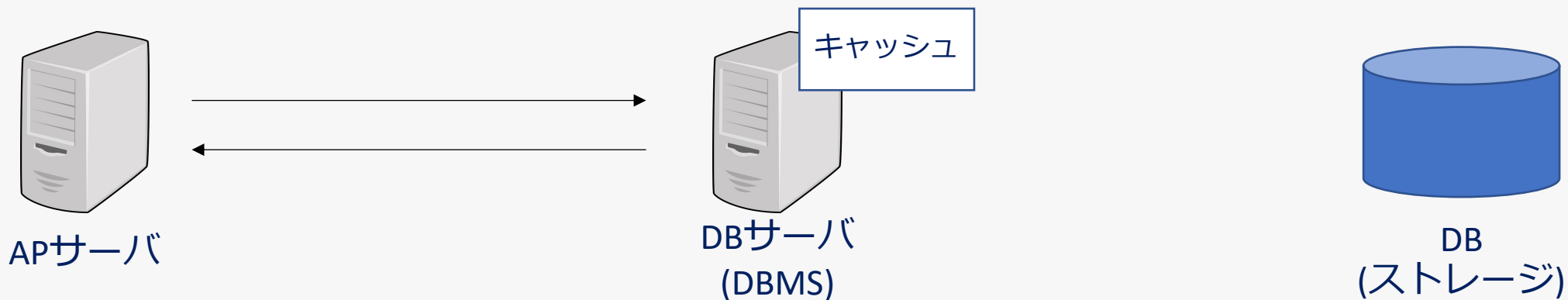
アーカイブログ・・・バックアップした状態からのデータベースの更新内容を記録するログ。
ロールフォワードに利用される

性能改善（キャッシュ）

DBサーバの**キャッシュ（メモリ）**上にデータがない場合、ストレージへのアクセスが発生するため、性能は劣化する



DBサーバの**キャッシュ（メモリ）**上にデータがある場合、ストレージへのアクセスが発生しないため、性能は向上する



豆知識: DB再起動直後は、キャッシュにデータが乗っていないので、あえてリクエストを投げてSQLを実行しキャッシュにデータを乗せる処置をとることもある（暖機運転）

性能改善（索引（インデックス））

データを取得する際に、**索引(インデックス)**を利用すると高速でデータを取得できます。

例) 1万件くらいデータのあるstudentsテーブルから、名まえが山田孝之という人を取得する場合、ディスクを全部見るよりもインデックスを見て、山田孝之がディスクのどこにあるのかわかって取得したほうが速い

*) 主キーには初めからインデックスが張られています。他の列にインデックスを張るには手動で作成する必要があります (CREATE INDEX)。よく検索する列（WHEREで絞り込むなど）にはインデックスを張ることも検討しましょう

豆知識: 何でもかんでもインデックス使うというのは良くない。例えば、山田で始まる人をstudentsテーブルから検索する場合に、studentsテーブルに山田が1%しかいない場合には、インデックスを使う方が速いが、山田が80%存在する場合はインデックスで1つ1つデータを取ってくるより、studentsテーブル内のデータを一括で取ってきてから絞り込む方が速い。

目安として、絞り込む対象が全データ量の15%未満の場合はインデックスを利用したほうが速い

(Oracleのマニュアル参照)

チューニングの際に、インデックスを張るかどうかはデータ量と検索する内容次第ということは覚えておきましょう！！

性能改善（アクセスパス(実行計画)、オプティマイザ）

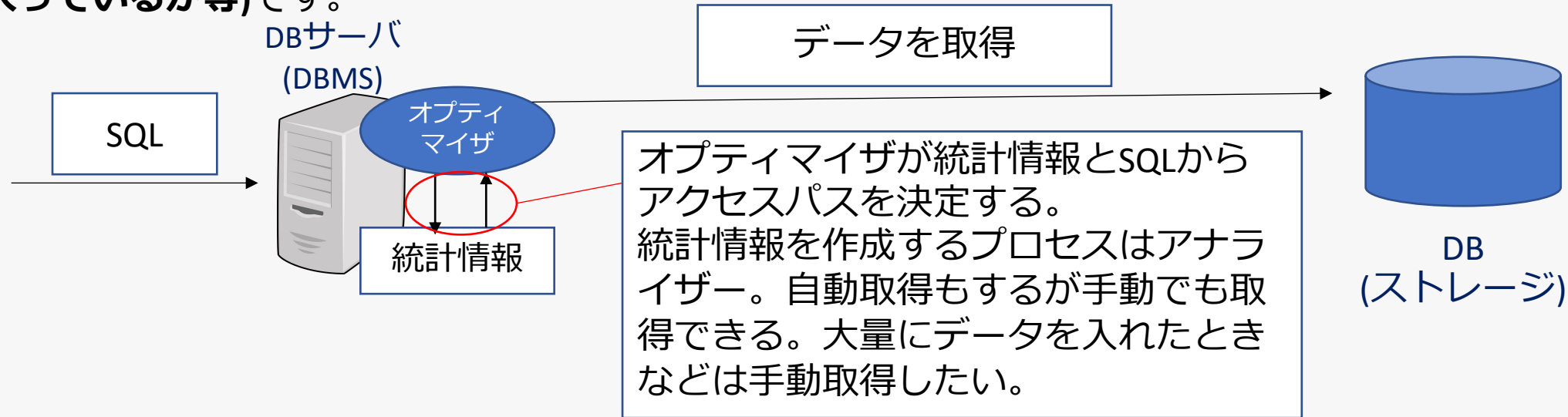
SQLを実行しても、データベースの状態、テーブル構造などで、データを取得する方法（**アクセスパス（実行計画）**）は異なります。

例) インデックスを利用するのか、しないのか

例) データを全部持ってきて絞り込むのか、必要なデータだけ取ってくるのか
等

この、アクセスパス（実行計画）を決めるプロセスが**オプティマイザ**です。

オプティマイザがアクセスパスを決定するのに必要な情報が**統計情報(どれだけテーブルにデータが入っているか等)**です。



豆知識: 統計情報に基づいてアクセスパスを決めるオプティマイザを**コストベース**。所定のルールで実行計画を決めるのを**ルールベース**と言う。現在はほとんどコストベース

応用情報技術者令和元年秋期 午前問29

データベースに媒体障害が発生したときのデータベースの回復法はどれか。

ア.障害発生時，異常終了したトランザクションをロールバックする。

→誤り。これは、トランザクションの障害からの回復です

イ.障害発生時点でコミットしていたがデータベースの実更新がされていないトランザクションをロールフォワードする。

→誤り。これは、トランザクションの障害時のチェックポイントからの回復です

ウ.障害発生時点でまだコミットもアボートもしていなかった全てのトランザクションをロールバックする。

→誤り。これは、トランザクションの障害時のチェックポイントからの回復です

エ.バックアップコピーでデータベースを復元し，バックアップ取得以降にコミットした全てのトランザクションをロールフォワードする。

→正しい。媒体障害ではバックアップから回復して、ロールフォワードします

応用情報技術者令和元年秋期 午前問30

分散トランザクション管理において，複数サイトのデータベースを更新する場合に用いられる2相コミットプロトコルに関する記述のうち，適切なものはどれか。

ア.主サイトが一部の従サイトからのコミット準備完了メッセージを受け取っていない場合，コミット準備が完了した従サイトに対してだけコミット要求を発行する。

→誤り。コミット準備完了を全従サイトから受け取ったのち、コミット要求をします

イ.主サイトが一部の従サイトからのコミット準備完了メッセージを受け取っていない場合，全ての従サイトに対して再度コミット準備要求を発行する。

→誤り。コミット準備完了メッセージを受け取るまで待機します

ウ.主サイトが全ての従サイトからコミット準備完了メッセージを受け取った場合，全ての従サイトに対してコミット要求を発行する。

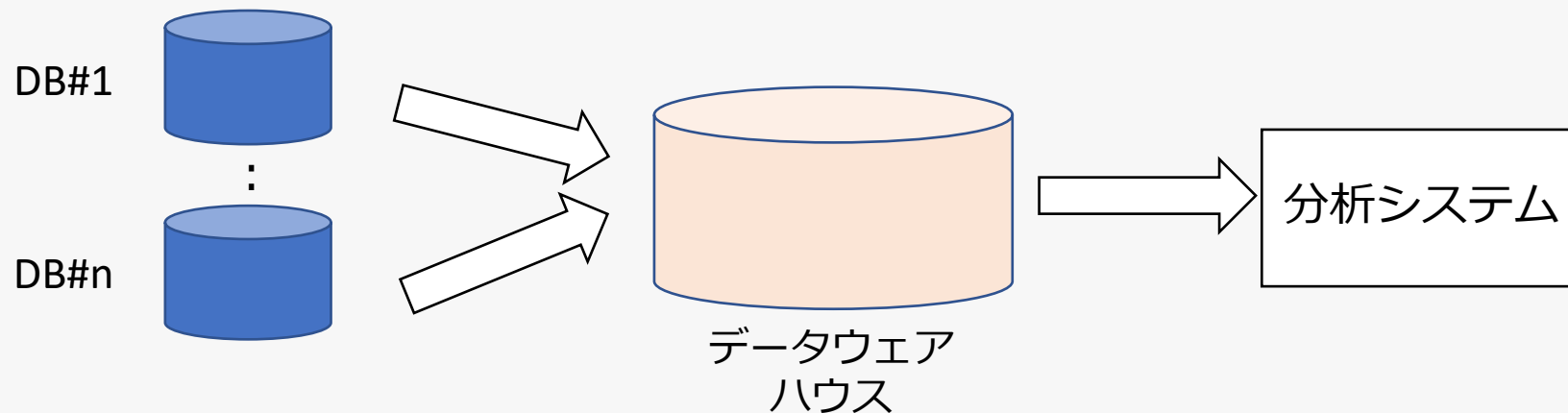
→正しい

エ.主サイトが全ての従サイトに対してコミット準備要求を発行した場合，従サイトは，コミット準備が完了したときだけ応答メッセージを返す。

→誤り。コミット準備が完了しない場合にも、主サイトにメッセージを返します

データウェアハウス

システム上のデータを統合して時系列に蓄積するデータベース。ビッグデータの分析に用いられる。



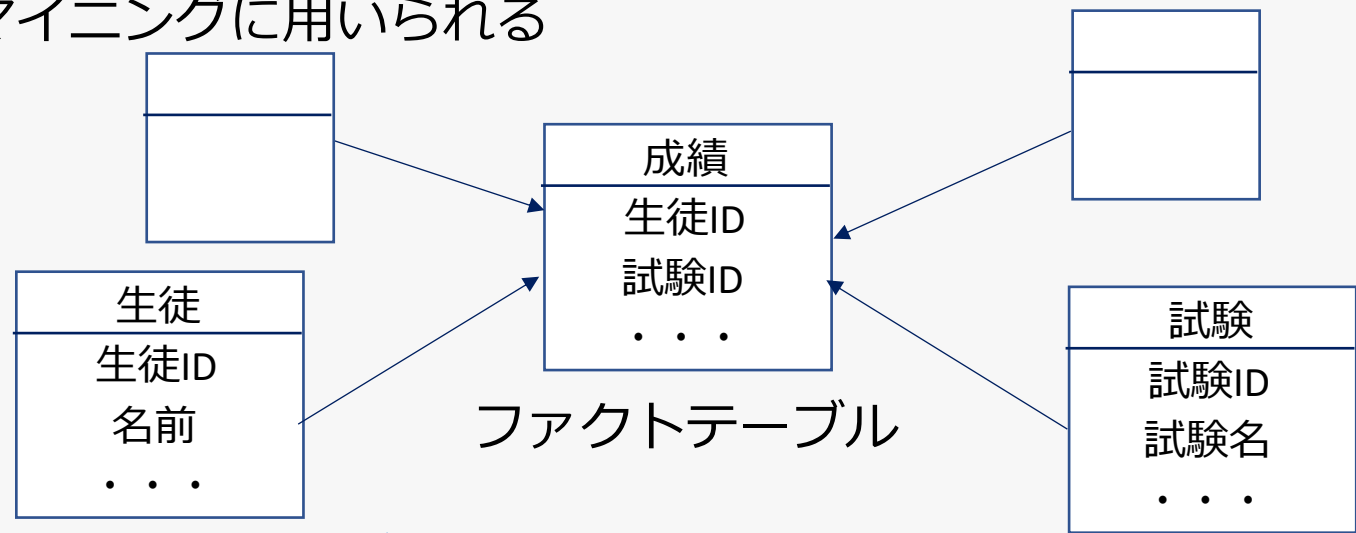
データレイク

構造化(列や行で表現して整理したデータ)、非構造化(構造化されていないデータ。文書、音声、画像など)されたデータをそのままの形で蓄積するデータベース。

構造化されたデータを保存する → **データウェアハウス**

構造化、非構造化データを保存する → **データレイク**

スタースキーマ: データウェアハウスの構造で、**ファクトテーブル**とファクトテーブルに紐づいた**ディメンジョンテーブル**が放射状に関連付けられる。ディメンジョンテーブルは、マスタデータを記録して、データマイニングに用いられる



データマイニング

膨大なデータの中から、未知の規則性、関係性を得る手法

テキストマイニング	アンケート、SNSなどの大量のテキストデータを解析して、商品の評価、業務の問題点を抽出する
クラスタリング	大量のデータの中から、類似した性質の持つデータのグループに分ける
クラス分類	クラスをあらかじめ定義して、データがどのクラスに分類されるのかを決定する
アソシエーションルール	大量のデータの中から相関関係を導き出す技術

ビッグデータ

膨大なデータの集合。標準的なデータベースでなく、クラウドコンピューティングや分散処理で解析する

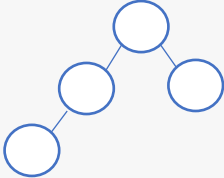
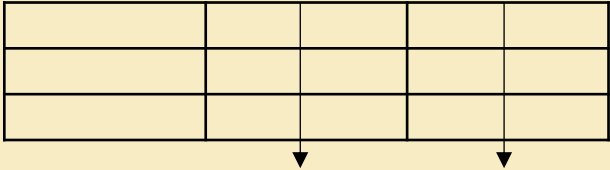
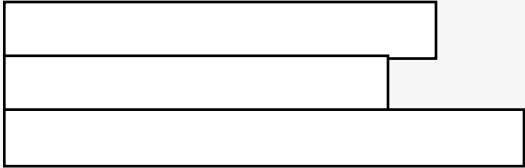
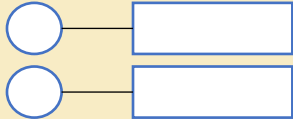
経営ダッシュボード	経営状況を様々な視点で分析する。売上推移、顧客層 等
パーソナルデータ	個人の購買情報やスマートフォンなどから収集した個人的なデータ。場合に応じて名前を変更する
オープンデータ	公的機関が保有する公開データ
レポーティング分析	ビッグデータを定められた手順で定期的に分析してレポートを作成する手法
アドホック分析	レポーティング分析からある項目を深く分析する手法

NoSQL(Not Only SQL)

データへのアクセスをSQLに限定しないデータベース管理システム。MongoDBなどで、関係データベース（RDBMS）やオブジェクト指向データベースは含まない

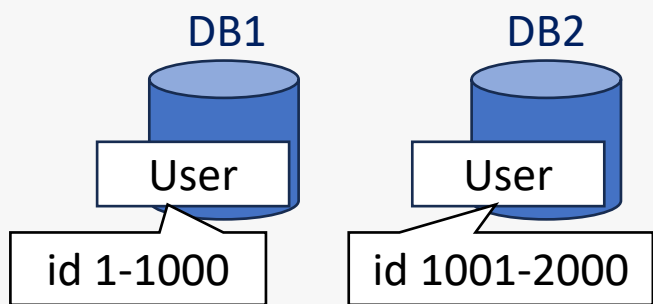
- ・ RDBMSのように、扱いやすい形でデータを格納することはできない。
- ・ パフォーマンスが高く、大規模なデータの扱いに適している。

NoSQLの種類

グラフ指向	データをグラフ構造で関連させるデータベース。 大量のデータを効率的に分析できる 例) Neptune, Neo4j	
列指向	列単位でデータを格納し、高速なアクセスを実現。大規模なデータ分析や集計処理に適している。 行の追加、削除は不得意 例) Cassandra, RedShift	
ドキュメント指向	自由な長さや構造のテキストデータを保存する。データの構造を1件ごとにかえることができる 例) MongoDB, DynamoDB	
KVS(Key Value Store)	キーと値のペアでデータを格納するシンプルなデータベース。スケーラビリティが高く、大量のデータを扱うのに適している。 例) Memcached, Redis	Key Value 

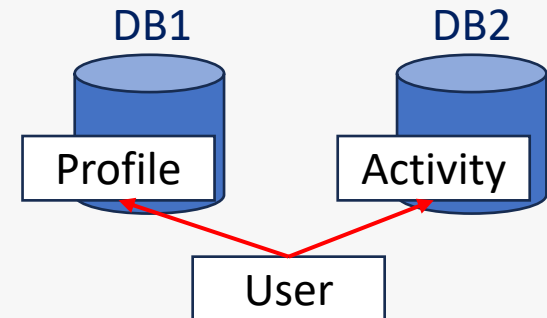
分散データベースの基本概念

複数のサイトに配置されたデータベースシステムを**分散データベース**という。データの分散や複製が利用者から見えないようにする**透過性**を意識して構築する。



水平分散（シャーディング）

データを行単位で分割して複数のノードに分散



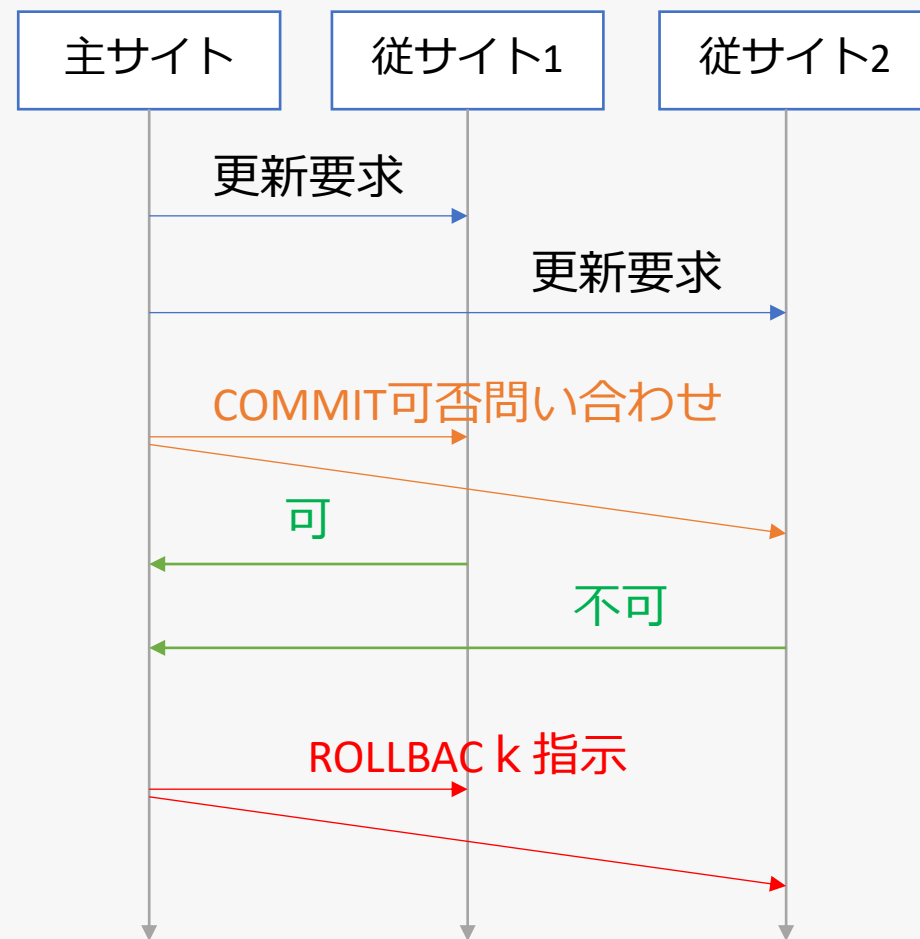
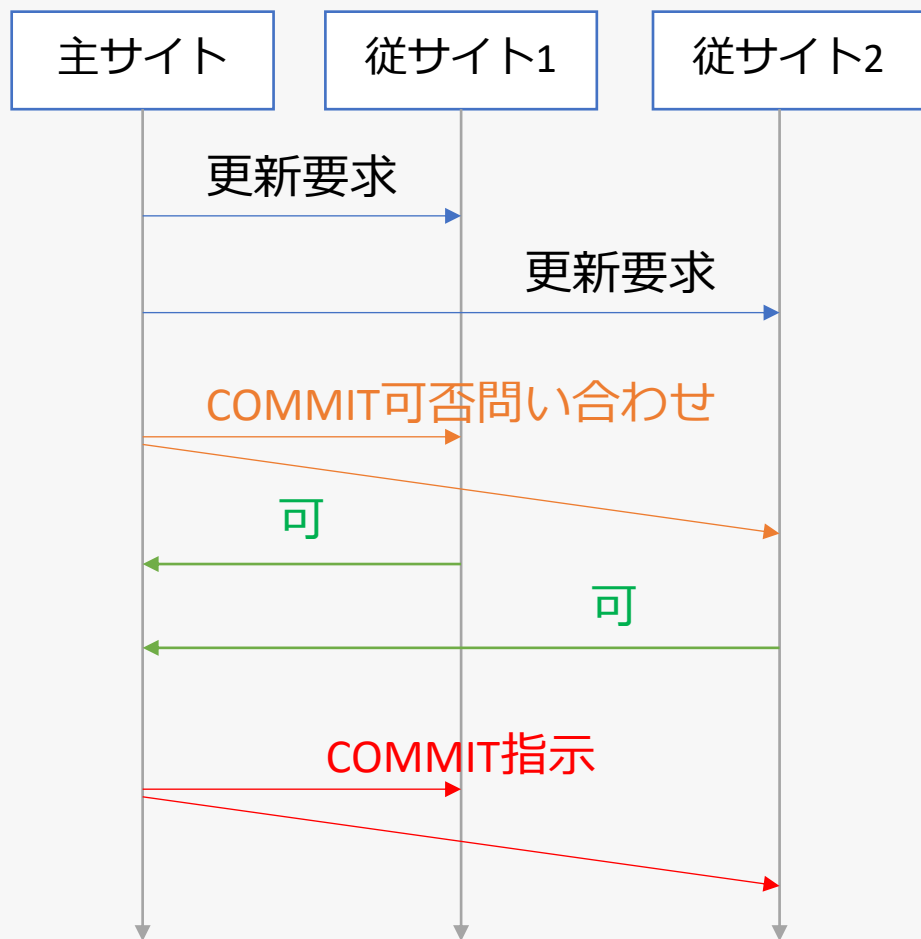
垂直分散

データを列単位で分割し、複数のノードに分散

レプリケーション	データの複製を複数のサイトに配置し、可用性と性能を向上させること
CAP定理	一貫性 （Consistency、データの整合性を保つこと）、 可用性 （Availability、利用したいときに利用できること）、 分断耐性 （Partition tolerance、複数サーバにデータを分散すること）の同時実現は不可能であり、同時には最大2つまでしか満たせないということ。
BASE特性	可用性とスケーラビリティを重視し、一貫性を緩和した特性を表す。Basically Available（基本的に利用可能）、Soft state（柔軟な状態）、Eventually consistent（最終的に一貫性がある）の略
結果整合性	分散システムにおいて、更新直後は古いデータが読み取られる可能性があるが、しばらくすると全ノードで更新後のデータが返される特性。一時的な不整合を許容することで、可用性とパフォーマンスを重視するシステムに適している。

2層コミットメント

データベースが2つの場所にまたがっている場合には複雑な制御をする。これを2層コミットメントという。**主サイト（調停者）**と**従サイト（参加者）**のそれぞれが存在し、全ての従サイトがコミット可能ならコミット、それ以外はロールバックを主サイトが従サイトに支持する



応用情報技術者平成30年春期 午前問30

ビッグデータの基盤技術として利用されるNoSQLに分類されるデータベースはどれか。

ア.関係データモデルをオブジェクト指向データモデルに拡張し、操作の定義や型の継承関係の定義を可能としたデータベース

→誤り。オブジェクト指向データベースの説明でNoSQLには分類されないです

イ.経営者の意思決定を支援するために、ある主題に基づくデータを現在の情報とともに過去の情報も蓄積したデータベース

→誤り。データウェアハウスの説明で、NoSQLの説明ではないです

ウ.様々な形式のデータを一つのキーに対応付けて管理するキーバリュ型データベース

→正しい。NoSQLは、グラフ指向、列指向、ドキュメント指向、KVS(キーバリュ型)などがあります

エ.データ項目の名称や形式など、データそのものの特性を表すメタ情報を管理するデータベース

→誤り。リポジトリの説明です

基本情報技術者平成29年春期 午前問29

ビッグデータの活用例として、大量のデータから統計学的手法などを用いて新たな知識(傾向やパターン)を見つけ出すプロセスはどれか。

ア.データウェアハウス

→誤り。データウェアハウスは過去のデータを蓄積したデータベースのことです

イ.データディクショナリ

→誤り。データディクショナリはデータの属性や性質を保存します

ウ.データマイニング

→正しい。

エ.メタデータ

→誤り。メタデータは、データそのものでなくデータに付属する情報（列名、テーブル名）のことを言います

データ操作・・・選択、挿入、削除、射影、結合操作

SQL文・・・SELECT文、UPDATE文、INSERT文、DELETE文、WHERE句、DISTINCT、ORDER BY、GROUP BY、副問い合わせ、IN、EXISTS、べき等

Viewの作成、CREATE文（PRIMARY KEY, NOT NULL, UNIQUE, CHECK, FOREIGN KEY REFERENCES）

GRANT文, REVOKE文、SQLの関数（CONCAT、SUBSTRING、LENGTH、TRIM、COALESCE、ROUND、FLOOR、CEIL）

トランザクション処理・・・コミット、ロールバック

障害回復（トランザクション障害からの回復）・・・ログ(REDOログ、UNDOログ)、チェックポイント、ロールバック、ロールフォワード

障害回復（媒体障害からの回復）・・・バックアップファイル、アーカイブログ

性能改善・・・キャッシュ、索引（インデックス）、アクセスパス（実行計画）、オプティマイザ（コストベース、ルールベース）、統計情報

データウェアハウス、データレイク、スタースキーマ（ファクトテーブル、ディメンジョンテーブル）、データマイニング（テキストマイニング、クラスタリング、クラス分類、アソシエーションルール）

ビッグデータ・・・経営ダッシュボード、パーソナルデータ、オープンデータ、レポーティング分析、アドホック分析

NoSQL・・・グラフ指向、列指向、ドキュメント指向、KVS(Key Value Store)

分散データベース・・・水平分散、垂直分散、レプリケーション、CAP定理、BASE特性、結果整合性、2相コミットメント

過去問（データ操作、トランザクション
処理、データベース応用）

JSON形式で表現される図1、図2のような商品データを複数のWebサービスから取得し、商品データベースとして蓄積する際のデータの格納方法に関する記述のうち、適切なものはどれか。ここで、商品データの取得元となるWebサービスは随時変更され、項目数や内容は予測できない。したがって、商品データベースの検索時に使用するキーにはあらかじめ制限を設けない。

```
{
  "_id": "AA09",
  "品名": "47 型テレビ",
  "価格": "オープンプライス",
  "関連商品 id": [
    "AA101",
    "BC06"
  ]
}
```

図1 A社Webサービスの商品データ

```
{
  "_id": "AA10",
  "商品名": "りんご",
  "生産地": "青森",
  "価格": 100,
  "画像 URL": "http://www.example.com/apple.jpg"
}
```

図2 B社Webサービスの商品データ

ア. 階層型データベースを使用し、項目名を上位階層とし、値を下位階層とした2階層でデータを格納する。

イ. グラフデータベースを使用し、商品データの項目名の集合から成るノードと値の集合から成るノードを作り、二つのノードを関係付けたグラフとしてデータを格納する。

ウ. ドキュメントデータベースを使用し、項目構成の違いを区別せず、商品データ単位にデータを格納する。

エ. 関係データベースを使用し、商品データの各項目名を個別の列名とした表を定義してデータを格納する。

JSON形式で表現される図1、図2のような商品データを複数のWebサービスから取得し、商品データベースとして蓄積する際のデータの格納方法に関する記述のうち、適切なものはどれか。ここで、商品データの取得元となるWebサービスは随時変更され、項目数や内容は予測できない。したがって、商品データベースの検索時に使用するキーにはあらかじめ制限を設けない。

```
{
  "_id": "AA09",
  "品名": "47 型テレビ",
  "価格": "オープンプライス",
  "関連商品 id": [
    "AA101",
    "BC06"
  ]
}
```

図1 A 社 Web サービス
の商品データ

```
{
  "_id": "AA10",
  "商品名": "りんご",
  "生産地": "青森",
  "価格": 100,
  "画像 URL": "http://www.example.com/apple.jpg"
}
```

図2 B 社 Web サービスの商品データ

ア. 階層型データベースを使用し、項目名を上位階層とし、値を下位階層とした2階層でデータを格納する。
→誤り。階層型データベースは、データを木構造で表現するのに適しているが、商品データの項目数や内容が予測できない場合には柔軟性に欠ける。

イ. グラフデータベースを使用し、商品データの項目名の集合から成るノードと値の集合から成るノードを作り、二つのノードを関係付けたグラフとしてデータを格納する。
→誤り。グラフデータベースは、ノード間の関係性を表現するのに優れているが、商品データのような単純な構造のデータを格納するには過剰な機能を持っている。

ウ. **ドキュメントデータベースを使用し、項目構成の違いを区別せず、商品データ単位にデータを格納する。**
→正しい。ドキュメントデータベースは、JSONなどの柔軟な形式でデータを格納できるため、項目数や内容が予測できない商品データの格納に適している。また、各商品データを独立したドキュメントとして扱えるため、検索時のキーに制限を設ける必要がない。

エ. 関係データベースを使用し、商品データの各項目名を個別の列名とした表を定義してデータを格納する。
→誤り。構造化されたデータの格納に適しているが、項目数や内容が予測できない場合には、スキーマの変更が頻繁に必要となり、運用が難しくなる。

「出典：応用情報技術者令和6年春期 午前問26」

“部品”表及び“在庫”表に対し、SQL文を実行して結果を得た。SQL文の[a]に入れる字句はどれか。

部品

部品 ID	発注点
P01	100
P02	150
P03	100

在庫

部品 ID	倉庫 ID	在庫数
P01	W01	90
P01	W02	90
P02	W01	150

〔結果〕

部品 ID	発注要否
P01	不要
P02	不要
P03	必要

```
SELECT 部品.部品ID AS 部品ID,  
       CASE WHEN 部品.発注点 > [ a ]  
         THEN N'必要' ELSE N'不要' END AS 発注要否  
FROM 部品 LEFT OUTER JOIN 在庫  
  ON 部品.部品ID = 在庫.部品ID  
GROUP BY 部品.部品ID, 部品.発注点;
```

ア. COALESCE(MIN(在庫.在庫数), 0)

イ. COALESCE(MIN(在庫.在庫数), NULL)

ウ. COALESCE(SUM(在庫.在庫数), 0)

エ. COALESCE(SUM(在庫.在庫数), NULL)

「出典：応用情報技術者令和6年春期 午前問26」

“部品”表及び“在庫”表に対し、SQL文を実行して結果を得た。SQL文の[a]に入れる字句はどれか。

部品

部品 ID	発注点
P01	100
P02	150
P03	100

在庫

部品 ID	倉庫 ID	在庫数
P01	W01	90
P01	W02	90
P02	W01	150

〔結果〕

部品 ID	発注要否
P01	不要
P02	不要
P03	必要

```
SELECT 部品.部品ID AS 部品ID,  
       CASE WHEN 部品.発注点 > [ a ]  
         THEN N'必要' ELSE N'不要' END AS 発注要否  
FROM 部品 LEFT OUTER JOIN 在庫  
  ON 部品.部品ID = 在庫.部品ID  
GROUP BY 部品.部品ID, 部品.発注点;
```

ア. COALESCE(MIN(在庫.在庫数), 0)

イ. COALESCE(MIN(在庫.在庫数), NULL)

ウ. COALESCE(SUM(在庫.在庫数), 0)

エ. COALESCE(SUM(在庫.在庫数), NULL)

→ウ.が正解

SQL文の目的は、“部品”表と“在庫”表を結合し、各部品について**発注の要否を判定すること**です。発注要否は、**部品の発注点と在庫数の合計を比較して**決定されます。

COALESCE関数は、引数のリストの中から最初の**NULL以外の値を返します**。アとイは、MIN関数を使用しています。これでは、各部品の最小在庫数が取得されるだけで、**全体の在庫数の合計は求められません**。

エは、SUM関数を使用していますが、第2引数がNULLです。これでは、**在庫数の合計がNULLの場合、比較ができなくなってしまう**。

ウは、SUM関数で在庫数の合計を求め、COALESCEの第2引数で0を指定しています。これにより、在庫数の合計がNULLの場合は0に置き換えられ、発注点との比較が可能になります。

「出典：応用情報技術者令和5年春期 午前問28」

データベースシステムの操作の説明のうち、ベキ等(idempotent)な操作の説明はどれか。

ア. 同一の操作を複数回実行した結果と、一回しか実行しなかった結果が同一になる操作

イ. トランザクション内の全ての処理が成功したか、何も実行されなかったかのいずれかの結果にしかない操作

ウ. 一つのノードへのレコードの挿入処理を、他のノードでも実行する操作

エ. 複数のトランザクションを同時に実行した結果と、順番に実行した結果が同一になる操作

「出典：応用情報技術者令和5年春期 午前問28」

データベースシステムの操作の説明のうち、ベキ等(idempotent)な操作の説明はどれか。

ア. 同一の操作を複数回実行した結果と、一回しか実行しなかった結果が同一になる操作

正しい。ベキ等(idempotent)な操作とは、同一の操作を複数回実行しても、一回実行した場合と同じ結果になる操作のことを指します。

イ. トランザクション内の全ての処理が成功したか、何も実行されなかったかのいずれかの結果にしかない操作

誤り。トランザクションのACID特性における「原子性(Atomicity)」を表しています。

ウ. 一つのノードへのレコードの挿入処理を、他のノードでも実行する操作

誤り。分散データベースシステムにおけるレプリケーション(複製)の一種である「マルチマスターレプリケーション」を表しています。

エ. 複数のトランザクションを同時に実行した結果と、順番に実行した結果が同一になる操作

誤り。トランザクションの「シリアライズビリティ(Serializability)」を表しています。

「出典：応用情報技術者令和4年春期 午前問26」

CAP定理におけるAとPの特性をもつ分散システムの説明として、適切なものはどれか。

ア. 可用性と整合性と分断耐性の全てを満たすことができる。

イ. 可用性と整合性を満たすが分断耐性を満たさない。

ウ. 可用性と分断耐性を満たすが整合性を満たさない。

エ. 整合性と分断耐性を満たすが可用性を満たさない。

「出典：応用情報技術者令和4年春期 午前問26」

CAP定理におけるAとPの特性をもつ分散システムの説明として、適切なものはどれか。

ア. 可用性と整合性と分断耐性の全てを満たすことができる。

イ. 可用性と整合性を満たすが分断耐性を満たさない。

ウ. 可用性と分断耐性を満たすが整合性を満たさない。

正しい。CAP定理は、分散システムにおいて「整合性(Consistency)」「可用性(Availability)」「分断耐性(Partition tolerance)」の3つの特性を同時に満たすことは不可能であると述べています。その中で、「可用性」と「分断耐性」を満たすシステムは、「整合性」を犠牲にすることになります。

エ. 整合性と分断耐性を満たすが可用性を満たさない。

「出典：応用情報技術者令和元年秋期 午前問27」

図のような関係データベースの"注文"表と"注文明細"表がある。"注文"表の行を削除すると、対応する"注文明細"表の行が、自動的に削除されるようにしたい。SQL文のON DELETE句に指定する語句はどれか。ここで、図中の実線の下線は主キーを、破線の下線は外部キーを表す。

注文

<u>注文番号</u>	注文日	<u>顧客番号</u>
-------------	-----	-------------

注文明細

<u>注文番号</u>	<u>明細番号</u>	<u>商品番号</u>	数量
-------------	-------------	-------------	----

ア. CASCADE

イ. INTERSECT

ウ. RESTRICT

エ. UNIQUE

「出典：応用情報技術者令和元年秋期 午前問27」

図のような関係データベースの"注文"表と"注文明細"表がある。"注文"表の行を削除すると、対応する"注文明細"表の行が、自動的に削除されるようにしたい。SQL文のON DELETE句に指定する語句はどれか。ここで、図中の実線の下線は主キーを、破線の下線は外部キーを表す。

注文

<u>注文番号</u>	注文日	<u>顧客番号</u>
-------------	-----	-------------

注文明細

<u>注文番号</u>	<u>明細番号</u>	<u>商品番号</u>	数量
-------------	-------------	-------------	----

ア. CASCADE

→ON DELETEにCASCADEを指定すると対応する行も自動的に削除されます

イ. INTERSECT

→INTERSECTは集合演算子で、積集合を求めるのに用います

ウ. RESTRICT

→RESTRICTは参照制約を指定するのに用いられます。

エ. UNIQUE

→UNIQUEはUNIQUE制約を指定するのに用いられます。

「出典：基本情報技術者平成26年春期 午前問28」

"商品"表, "在庫"表に対する次のSQL文と, 同じ結果が得られるSQL文はどれか。ここで, 下線部は主キーを表す。

SELECT 商品番号 FROM 商品 WHERE 商品番号 NOT IN (SELECT 商品番号 FROM 在庫)

商品			在庫		
<u>商品番号</u>	商品名	単価	<u>倉庫番号</u>	<u>商品番号</u>	在庫数

ア. SELECT 商品番号 FROM 在庫 WHERE EXISTS
(SELECT 商品番号 FROM 商品)

イ. SELECT 商品番号 FROM 在庫 WHERE NOT EXISTS
(SELECT 商品番号 FROM 商品)

ウ. SELECT 商品番号 FROM 商品 WHERE EXISTS
(SELECT 商品番号 FROM 在庫 WHERE 商品.商品番号 = 在庫.商品番号)

エ. SELECT 商品番号 FROM 商品 WHERE NOT EXISTS
(SELECT 商品番号 FROM 在庫 WHERE 商品.商品番号 = 在庫.商品番号)

「出典：基本情報技術者平成26年春期 午前問28」

"商品"表, "在庫"表に対する次のSQL文と, 同じ結果が得られるSQL文はどれか。ここで, 下線部は主キーを表す。

SELECT 商品番号 FROM 商品 WHERE 商品番号 NOT IN (SELECT 商品番号 FROM 在庫)

商品			在庫		
<u>商品番号</u>	商品名	単価	<u>倉庫番号</u>	<u>商品番号</u>	在庫数

ア. SELECT 商品番号 FROM 在庫 WHERE EXISTS
(SELECT 商品番号 FROM 商品)

イ. SELECT 商品番号 FROM 在庫 WHERE NOT EXISTS
(SELECT 商品番号 FROM 商品)

ウ. SELECT 商品番号 FROM 商品 WHERE EXISTS
(SELECT 商品番号 FROM 在庫 WHERE 商品.商品番号 = 在庫.商品番号)

エ. SELECT 商品番号 FROM 商品 WHERE NOT EXISTS
(SELECT 商品番号 FROM 在庫 WHERE 商品.商品番号 = 在庫.商品番号)

→エが正解

商品.商品番号 NOT IN (SELECT 商品番号 FROM 在庫)
は在庫に存在する商品番号以外の商品に絞り込んでいます

商品 NOT EXISTS(SELECT 商品番号 FROM 在庫 WHERE 商品.商品
番号 = 在庫.商品番号)も同様に在庫に存在しない商品だけ選
択されます

「出典：応用情報技術者平成31年春期 午前問29」

ビッグデータのデータ貯蔵場所であるデータレイクの特徴として、適切なものはどれか。

ア. あらゆるデータをそのままの形式や構造で格納しておく。

イ. データ量を抑えるために、データの記述情報であるメタデータは格納しない。

ウ. データを格納する前にデータ利用方法を設計し、それに沿ってスキーマをあらかじめ定義しておく。

エ. テキストファイルやバイナリデータなど、格納するデータの形式に応じてリポジトリを使い分ける。

「出典：応用情報技術者平成31年春期 午前問29」

ビッグデータのデータ貯蔵場所であるデータレイクの特徴として、適切なものはどれか。

ア. あらゆるデータをそのままの形式や構造で格納しておく。

→正しい。データレイクの説明です

イ. データ量を抑えるために、データの記述情報であるメタデータは格納しない。

→誤り。メタデータも含めて保存します

ウ. データを格納する前にデータ利用方法を設計し、それに沿ってスキーマをあらかじめ定義しておく。

→誤り。データのスキーマを定義せずそのままの形式で保存します

エ. テキストファイルやバイナリデータなど、格納するデータの形式に応じてリポジトリを使い分ける。

→データの形式で分けずに混在させて保存します

「出典：基本情報技術者令和元年秋期 午前問28」

一つのトランザクションはトランザクションを開始した後、五つの状態(アクティブ, アボート処理中, アボート済, コミット処理中, コミット済)を取り得るものとする。このとき、取ることのない状態遷移はどれか。

	遷移前の状態	遷移後の状態
ア	アボート処理中	アボート済
イ	アボート処理中	コミット処理中
ウ	コミット処理中	アボート処理中
エ	コミット処理中	コミット済

「出典：基本情報技術者令和元年秋期 午前問28」

一つのトランザクションはトランザクションを開始した後、五つの状態(アクティブ, アボート処理中, アボート済, コミット処理中, コミット済)を取り得るものとする。このとき、取ることのない状態遷移はどれか。

	遷移前の状態	遷移後の状態
ア	アボート処理中	アボート済
イ	アボート処理中	コミット処理中
ウ	コミット処理中	アボート処理中
エ	コミット処理中	コミット済

→正解はイ

ア: アボート処理が成功すると、アボート処理中からアボート済に移動します

イ: アボート処理中からコミット処理中には移動しません

ウ: コミット処理が失敗すると、コミット処理中からアボート処理中に移動します

エ: コミット処理が成功すると、コミット処理中からコミット済に移動します